

Figure D.2 JSONP uses a `<script>` tag to send cross-origin requests.

The following listing shows how to use JSONP to recreate the Flickr example from chapter 1.

Listing D.1 Using JSONP to access Flickr

```
<!DOCTYPE html>
<html>
<body>
<div id="photos"></div>
<script>
function loadPhotos(data) {
  if (data.stat == 'ok') {
    var photos = data.photos.photo;
    for (var i = 0; i < photos.length; i++) {
      var img = document.createElement('img');
      img.src = photos[i].url_q;
      document.getElementById('photos').appendChild(img);
    }
  } else {
    alert(data.message);
  }
};
</script>
<script src="https://api.flickr.com/services/rest/
     ?method=flickr.people.getPublicPhotos&user_id=32951986%40N05&extras=url_
     q&format=json&jsoncallback=loadPhotos&api_key=<YOUR API KEY HERE>"></
     script>
</body>
</html>
```

Callback function that handles response.

Request is made via a <script> tag.

In this listing, the `XMLHttpRequest` is replaced with a `<script>` tag that initiates a request to the Flickr API server. The `<script>` tag can be created either by embedding a `<script>` tag in the HTML, or by dynamically creating it with JavaScript (by using `document.createElement('script')`). A `<script>` tag traditionally points to a file that contains JavaScript code. But in this case, the Flickr API server responds with code that wraps the API response in a JavaScript function. When the browser receives the response, it executes the JavaScript code, which in turn executes the callback function. Figure D.3 shows what a typical JSONP flow looks like.

The key to a successful JSONP request is the callback function. When initiating a JSONP request, the `<script>` tag has a parameter set to the name of the callback function. In figure D.3, the callback function is named `loadPhotos`. When sending a response back to the client, the server first pads the response with the name of the